

AI Career Roadmap & RAG/Agent Toolkit

A practical guide for an AI student building toward a career as an AI/ML Engineer, LLM/GenAI Engineer, or AI Solutions Engineer — starting with a RAG (Retrieval-Augmented Generation) project and extending toward AI Agents.

Context / How to use this document: I am an AI student with basic Python and ML knowledge. I want to build a portfolio-grade project involving a backend server, an API, a vector database, and an LLM — specifically a Retrieval-Augmented Generation (RAG) system that I can later extend into an AI Agent. Please use the roadmap and tool list below to help me set up my development environment, plan the build order, and write code step by step.

1. Career Context

Target roles this project builds toward: AI/ML Engineer, LLM/GenAI Engineer, AI Solutions Engineer / Consultant, Backend Engineer with AI specialization, and (as a freelance option) custom chatbot/RAG builder for small and medium businesses.

Learning path: (1) Master a RAG system end-to-end — ingestion, embeddings, vector storage, retrieval, generation, and API exposure. (2) Extend it into a simple AI Agent that uses RAG as one tool among others (e.g., web search, calculations). (3) Containerize and deploy it so it is a live, working demo — not just a notebook.

2. Target Project Architecture (RAG System)

Documents (PDFs/text) → Chunking → Embeddings → Vector Database → Retriever → LLM (with retrieved context) → FastAPI endpoint → Client (Streamlit UI, web widget, or WhatsApp/webhook integration). Everything should run locally first, then be containerized with Docker, then optionally deployed to a free-tier host.

3. Core Tools & Why Each Is Needed

Category	Tool	Why it's needed
Language	Python 3.10+	Primary language for almost all AI/ML and backend development; has the richest AI ecosystem.
Env/Packages	pip / venv (or conda)	Manage isolated project dependencies so packages don't conflict across projects.
Version Control	Git + GitHub	Track code changes and host the portfolio project publicly with a clear README.
Backend Framework	FastAPI	Exposes the RAG pipeline as an HTTP API (e.g. POST /ask); async, fast, auto-generates API docs.
App Server	Uvicorn	ASGI server that actually runs the FastAPI application.
LLM Provider	OpenAI API or Anthropic API	Supplies the generation step: takes retrieved context + user question, returns an answer.

Category	Tool	Why it's needed
Local LLM (optional)	Ollama	Runs open-source models (e.g. Llama, Mistral) locally, avoiding API costs during development.
Embeddings	sentence-transformers (or provider embeddings)	Converts text chunks into numeric vectors so semantic similarity search is possible.
Vector Database	ChromaDB	Beginner-friendly, runs locally, stores embeddings and retrieves the most relevant chunks.
Vector DB (scale-up)	Pinecone / Qdrant / Weaviate	Cloud-hosted alternatives once the project needs to scale beyond local storage.
Orchestration	LangChain	Wires together loaders, chunkers, embeddings, vector store, and LLM calls with less boilerplate.
Orchestration (RAG-focused)	LlamaIndex	Alternative to LangChain, specialized specifically for retrieval-augmented pipelines.
Agent Framework (next step)	LangGraph or CrewAI	Used after RAG is working, to add multi-step reasoning and multiple tools/agents.
Relational Database	PostgreSQL + SQLAlchemy	Stores structured data such as users, chat history, or logs alongside the vector store.
Containerization	Docker (+ Docker Compose)	Packages the app and its dependencies so it runs identically anywhere; required for real deployment.
Deployment (free tier)	Hugging Face Spaces / Render / Railway	Hosts the working demo publicly so it can be shared in a portfolio or with employers.
Dev Tools	VS Code, Postman, python-dotenv	Code editor, manual API testing, and safe storage of API keys via .env files.

4. Recommended Setup & Learning Order

- 1 Python + FastAPI — get a minimal API running ("Hello World" endpoint) before adding AI.
- 2 OpenAI or Claude API — learn to send a prompt and receive a response programmatically.
- 3 ChromaDB — store a few text chunks and retrieve them by similarity search.
- 4 LangChain (or LlamaIndex) — connect document loading, chunking, embeddings, vector store, and LLM into one pipeline.
- 5 Docker — containerize the working app once the core pipeline runs locally.
- 6 Deployment — push the container to a free host (Hugging Face Spaces / Render) for a public demo.
- 7 Agent extension — once RAG works, wrap it as a "tool" inside a simple agent (LangGraph/CrewAI) that can also call a web search tool.

5. Request to the Assisting AI

Please help me, step by step, to: (1) set up a local development environment with the tools above, (2) scaffold a FastAPI project structure for a RAG system, (3) write the ingestion pipeline (load documents, chunk, embed, store in ChromaDB), (4) write the retrieval + generation endpoint, (5) add a minimal Streamlit or HTML client to

test it, and (6) containerize the result with Docker. Please explain each step briefly as we go, and prefer minimal, working code over large amounts of boilerplate.

Generated as a personal learning/reference document for planning an AI engineering portfolio project.